

ExecuTorch Deep-Dive: A Technical Briefing for the EdgeMind Talk

This briefing is structured around the 13 topics you listed. For each, you'll find an engineer-level technical answer with the concrete API names, file/format details, version data, and numbers you can fold directly into Q&A pairs at the MCP Dev Summit. Where the public record is thin or where something is "experimental," that is called out explicitly so you don't get caught flat-footed.

1. ExecuTorch Architecture: `.pte` Format, Execution Plan, Runtime

ExecuTorch's runtime is responsible for two things: (1) loading a binary `.pte` program file generated by the ahead-of-time (AOT) `to_executorch()` step, and (2) executing the lowered model's instruction stream. The runtime is intentionally tiny — the core runtime library is **under 50 KB** when built without kernels or backends — so that it can drop into bare-metal, threadless, heapless embedded environments as well as modern smartphones.

`(PyTorch + 2)`

The `.pte` file format. A `.pte` file is a modified binary **FlatBuffer** with optional appended data segments. The structure is: `(PyTorch)`

- Bytes `[0..3]`: a `uint32_t` little-endian byte offset to the start of the FlatBuffer root table. `(PyTorch)`
- Bytes `[4..7]`: a magic string beginning with `ET` followed by two ASCII decimal digits identifying the format version. `(PyTorch)`
- Bytes `[8..]`: an optional extended header, recognized by a magic string beginning with `eh` and two ASCII decimal digits. The extended header includes the size of the

FlatBuffer-encoded core program data and the starting offset of any segments that follow. (PyTorch)

- After the headers comes the FlatBuffer program data (schema: `executorch/schema/program.fbs`). (PyTorch)
- After the program comes zero or more **segments**, typically aligned to **4096 bytes** (or another power-of-two matching the target's page size), so segments can be `mmap()`'d. Segments hold the large blobs: constant tensor weights, delegate-compiled blobs, etc. Offsets in `Program.segments` are relative to the segment offset embedded in the extended header. (PyTorch) (PyTorch)

This split matters because **program data sticks around for the lifetime of the model, while segment data is often freeable after init** — backend delegates can unload their precompiled data after `init()` completes, dropping peak RAM. Constant tensors point directly into the `.pte` data, avoiding an extra copy when the file is `mmap()`'d. (PyTorch) (PyTorch)

Execution plan and runtime components. When you call

`Module::load_method("forward")` (or the lower-level `program->load_method()`), the runtime:

1. Parses the FlatBuffer Program and the chosen Method.
2. Allocates the **Method Allocator** (a user-provided `MemoryAllocator`) for runtime structures: `Tensor` metadata, the chain of instructions, EValues, kernel/delegate handles, and other transient init-time state.
3. Allocates one or more **memory-planned buffers** (the "planned memory" arenas) — these are the static, ahead-of-time-sized arenas into which intermediate mutable tensor data lives. The runtime exposes how many arenas it needs via `method_meta->num_memory_planned_buffers()` and the byte size of each via `method_meta->memory_planned_buffer_size(id)`. You wrap them in a `HierarchicalAllocator` (which is why ExecuTorch supports multi-tier memory — e.g., SRAM + DRAM on a Hexagon DSP). (PyTorch) (PyTorch)

4. Walks the **chain of instructions** in the Method. Each instruction is either a kernel call (looked up by name in the kernel registry) or a `executorch_call_delegate` to a backend.
5. Returns the `Method` handle. Subsequent `method.execute()` calls just walk that pre-baked instruction list — there's no graph interpreter doing tracing or planning at runtime.

The major runtime modules under `executorch/runtime/` are: `core/` (EValue, error types, tagged-union value types), `executor/` (Program, Method, Method loading, instruction dispatch), `kernel/` (operator registry, kernel context, `executorch::aten::Tensor` — a source-compatible subset of `at::Tensor`), `backend/` (Backend interface, BackendRegistry, delegate dispatch), and `platform/` (thin portability shim for clock, abort, logging — what makes it work in RTOS or bare-metal). ExecuTorch deliberately avoids RTTI, exceptions, threads, and most templates in core code; these constraints are spelled out in `CONTRIBUTING.md` to keep binary size small and to support no-OS targets.

[GitHub](#) [GitHub](#)

2. ExecuTorch vs. ONNX Runtime Mobile, TFLite, MediaPipe LLM, llama.cpp, MLC-LLM

The most useful framing for your audience: **ExecuTorch is a PyTorch-native AOT compiler + tiny VM, not a graph interpreter.** Almost all the heavy graph work — quantization, partitioning, kernel selection, memory planning, delegate compilation — happens offline. The on-device runtime just executes a pre-baked instruction list.

- **vs. ONNX Runtime Mobile.** ORT Mobile runs ONNX graphs through a graph engine with Execution Providers (EPs) — CoreML EP, NNAPI EP, QNN EP, XNNPACK EP. It is broader in framework coverage (anything that can export to ONNX) and has substantially better Windows desktop story (DirectML, CUDA, TensorRT EPs).

ExecuTorch wins on (a) no intermediate format conversion — you go straight from `torch.export` to `.pte`, preserving PyTorch semantics including modern Python control flow that ONNX historically struggles with; (b) substantially smaller runtime (~50 KB core vs. multi-MB ORT Mobile binary); (c) tighter coupling with `torchao` quantization (group-wise int4/int8 weight-only, dynamic activation int8). ORT has a more mature on-device LLM Generate() API today. (Aman's AI Journal + 3)

- **vs. TFLite (LiteRT).** TFLite uses a FlatBuffer model + interpreter + optional delegates (NNAPI, Hexagon, GPU/OpenCL, Core ML). It's the most mature mobile runtime by tooling and ecosystem. Architecturally TFLite is closer to ExecuTorch than ORT (FlatBuffer + delegate model), but TFLite is TensorFlow-native and requires conversion from PyTorch via ONNX or `ai-edge-torch`. ExecuTorch is the "TFLite for PyTorch" — same shape, different framework. Public benchmarks vary; in one Samsung S21 image-classification micro-benchmark, TFLite ran at 23 ms / ~89 MB while PyTorch Mobile (the predecessor to ExecuTorch) ran at 38 ms / 126 MB — but PyTorch Mobile is *not* ExecuTorch, and ExecuTorch has closed most of that gap with XNNPACK + KleidiAI. (Aman's AI Journal + 3)
- **vs. MediaPipe LLM Inference API.** MediaPipe LLM is Google's high-level on-device LLM SDK (built on TFLite/XNNPACK and GPU under the hood) with a curated set of supported models (Gemma 3, Phi, Falcon, etc.). It's the easiest way to run a *supported* LLM on Android. ExecuTorch trades that turnkey simplicity for flexibility: you can export *any* PyTorch model, choose any backend partition, swap tokenizers, and integrate custom ops. If you want to run Gemma 3 on Android Pixel today with the least code, MediaPipe wins. If you need a custom model, custom LoRA, or NPU offload to Qualcomm HTP, ExecuTorch is the better story. (Awesome Agents)
- **vs. llama.cpp.** llama.cpp is a minimalist C/C++ runtime built around the GGUF format and a static transformer loop — there's no general graph engine, no partitioner, no autodiff. It is CPU-first (with growing CUDA/Metal/Vulkan kernels) and has by far the largest community of quantization formats (Q4_K_M, Q5_K_M, Q8_0, etc.). ExecuTorch beats llama.cpp on NPU/HTP offload, on model breadth (vision, ASR, multimodal — not just LLMs), and on PyTorch-native fidelity. llama.cpp beats

ExecuTorch on plug-and-play LLM support, breadth of quantization recipes, and lean CPU performance for niche transformer variants.

- **vs. MLC-LLM.** MLC-LLM compiles models through Apache TVM/Relax to many backends (Vulkan, Metal, OpenCL, CUDA, WebGPU). It is the strongest "compile-anywhere" LLM story, particularly for cross-vendor mobile GPUs. ExecuTorch's GPU story is narrower (Vulkan delegate, Metal/Core ML on iOS) but its CPU + NPU story is more turnkey if you're already in PyTorch and targeting Snapdragon, Apple, MediaTek, or Arm Ethos-U.

A nuance worth flagging in Q&A: one widely-cited 2025 paper ("Challenging GPU Dominance: When CPUs Outperform for On-Device LLM Inference," arXiv 2505.06461) lists ExecuTorch's chief outstanding challenge as "balancing a small memory footprint with execution speed" — a fair characterization of where the framework still trails llama.cpp on raw CPU LLM throughput in some regimes. [\(arxiv\)](#)

3. Quantization in ExecuTorch: int4/int8, GPTQ, AWQ, Group-wise

ExecuTorch's quantization story is now almost entirely delegated to `torchao` (the PyTorch-native quantization library, `pytorch/ao`). The integration is: you quantize a PyTorch `nn.Module` with a `torchao` config, then call `torch.export.export` → `to_edge_transform_and_lower` → `to_executorch`. Tensor subclasses in `torchao` represent the low-precision data types (INT4, INT8, FP8, MXFP4/6/8) and survive through export. [\(arxiv\)](#)

INT8 dynamic activation + INT4 weight (the canonical mobile recipe). For LLMs, the standard ExecuTorch quantization for mobile is **INT8-INT4**: 8-bit dynamic activation quantization on the linear layers, 4-bit weight-only quantization for the linear weights, with 8-bit token embeddings. Configured via `Int8DynamicActivationIntxWeightConfig` in `torchao.quantization`. This is what the official PyTorch-team `pytorch/Phi-4-mini-instruct-INT8-INT4` Hugging Face release uses. [\(Hugging Face\)](#)

Group-wise weight quantization. This is the central trick for keeping int4 accuracy acceptable: weights are partitioned along the input-channel dimension into groups (default **128 elements per group**), and each group gets its own scale and zero-point. For a Linear with shape `(out_features=4096, in_features=4096)` quantized to int4 with `group_size=128`:

- **Quantized weights:** `(out_features, in_features/2)` in uint8 (two 4-bit values packed per byte).
- **Scales:** `(out_features, in_features/128)` in fp16/bf16 — one scale per group.
- **Zero-points:** `(out_features, in_features/128)` in int4 or int8 — one per group.

Group size is the main lever: smaller groups (32, 64) preserve accuracy better but bloat the scale/zp metadata; larger groups (256) save space but lose accuracy. Group size 128 is the standard mobile sweet spot. Per-channel (one scale per output row) is the coarsest and degrades int4 LLM quality noticeably; per-token activation quantization is what makes the activation side viable at int8.

GPTQ / AWQ. `torchao` provides PTQ algorithms beyond plain Round-To-Nearest (RTN).

GPTQ (Optimal Brain Quantization, applied iteratively) is implemented via the

`MultiTensor` abstraction and module hooks — it does layer-by-layer error compensation using a small calibration set. **AWQ** (Activation-aware Weight Quantization) identifies the top ~1% salient weight channels (by activation magnitude) and scales them up before quantizing so they suffer less rounding error; on Phi-3 Mini, AWQ INT4 was shown to nearly match FP16 perplexity. Both are supported as flows in `torchao` and the resulting checkpoint can be lowered to ExecuTorch the same way as a plain RTN int4 model. `PyTorch`

`ONNX Runtime`

QAT. For very low bit-widths (especially int4), `torchao` provides quantization-aware training via the `quantize_` API. The PyTorch team explicitly recommends QAT over PTQ for sub-int8 LLMs on mobile. `PyPI`

Quality/size tradeoffs to cite in Q&A. For Llama 3.2 1B/3B quantized models on a OnePlus 12 vs. fp16: model size dropped 56%, memory dropped 41%, decode latency improved 2.5×, prefill 4.2×. The 1B model hit **50.2 tokens/s decode** and **260 tokens/s prefill** using XNNPACK + KleidiAI. A general rule-of-thumb for Q4_K_M-style int4 quantization is 1–3 MMLU points lost on average vs. fp16, model-dependent. [PyTorch](#) [Awesome Agents](#)

4. QAI Hub Integration, Snapdragon HTP, and SM8650

Qualcomm's backend in ExecuTorch is the **QNN** (Qualcomm Neural Network) delegate, living under `executorch/backends/qualcomm/`. It targets the Hexagon Tensor Processor (HTP, the NPU on Snapdragon SoCs); the same delegate technically also supports GPU and DSP backends through QNN, but HTP is the production path.

The export pipeline at a glance:

```
python
```

```

from executorch.backends.qualcomm.utils.utils import (
    generate_qnn_executorch_compiler_spec,
    generate_htp_compiler_spec,
    QcomChipset,
    to_edge_transform_and_lower_to_qnn,
)

backend_options = generate_htp_compiler_spec(
    use_fp16=not quantized,          # FP16 only on premium SoCs (8 G
    use_dlbc=False,                  # Deep Learning Bandwidth Compre
    use_multi_contexts=False,        # Share spill-fill across split
    performance_mode=QnnExecuTorchHtpPerformanceMode.kHtpBurst,
)

compile_spec = generate_qnn_executorch_compiler_spec(
    soc_model=QcomChipset.SM8650,    # Snapdragon 8 Gen 3
    backend_options=backend_options,
)

delegated = to_edge_transform_and_lower_to_qnn(
    quantized_model, example_inputs, compile_spec
)

executorch_program = delegated.to_executorch()
open("model_qnn.pt", "wb").write(executorch_program.buffer)

```

Partitioning. `QnnPartitioner` walks the exported graph and groups supported subgraphs into a single `executorch_call_delegate` node. Subgraphs delegated to HTP are converted into a QNN context binary embedded as a segment inside the `.pte`. Unsupported ops fall back to CPU (XNNPACK or portable kernels) — there is real, mixed HTP+CPU execution at runtime, but every crossing of the boundary costs latency, so good partitioning is what determines whether HTP is a win.

Op fusion and layout transforms. Two classes of optimization happen during lowering:

- **Op fusion:** patterns like Linear+BiasAdd, Conv+BN+ReLU, attention sub-blocks (matmul + softmax + matmul) are folded into single QNN ops where the HTP has hardware-fused kernels. This is done via `QnnPasses` in the partitioner before serialization.
- **Layout transforms:** the HTP prefers NHWC (channels-last) and specific tiling, while exported PyTorch graphs are typically NCHW (contiguous). The Qualcomm partitioner inserts permute/transpose ops at delegate boundaries, and a layout-canonicalization pass minimizes the number of inserted permutes (very similar in spirit to the XNNPACK channels-last pass). `PyTorch`

SM8650-specific optimizations. SM8650 = Snapdragon 8 Gen 3, which has **Hexagon v75** silicon. You target it by passing `QcomChipset.SM8650`. Required runtime libraries you ship in `jniLibs/arm64-v8a/`:

- `libQnnHtp.so` (HTP backend), `PyTorch`
- `libQnnHtpV75Stub.so` (the AArch64 stub for v75),
- `libQnnHtpV75Skel.so` (the Hexagon-side skeleton, pushed by stub at init),
- `libQnnSystem.so`. Additionally for ExecuTorch JNI you ship `libqnn_executorch_backend.so` linked against your `libexecutorch.so`. Notable SM8650 features the HTP exploits: increased VTCM (vector tightly-coupled memory) capacity, fp16 HTP support (premium tier only — pre-8 Gen 1 doesn't have it), and improved INT4/INT8 throughput per HMX (Hexagon Matrix Extensions) cycle.
`PyTorch` `PyTorch`

QAI Hub (`aihub.qualcomm.com`) is Qualcomm's hosted compile-and-profile service. It accepts a PyTorch `.pt` or ONNX file, compiles it to a QNN context binary (or DLC) targeting a specific device (`hub.Device("Snapdragon 8 Elite QRD")`, `"Samsung Galaxy S24 (Family)"`, etc.), and can run on real devices in their lab for profiling. The link-jobs feature combines multiple graphs into one context binary so weights are shared across sub-graphs — useful for prefill/decode pairs in an LLM. For ExecuTorch integration, the typical

pattern is: use QAI Hub to validate that your model compiles for the target SoC and to get profiling numbers, then use the local ExecuTorch QNN partitioner to produce a `.pte` for shipping. (As of the most recent docs, the AI Hub client supports up to 10 GB compressed uploads; recent additions include Snapdragon 8 Elite and Cockpit Gen 4 / SA8295P devices.) `Qualcomm`

5. ExecuTorch Training API: LoRA on-device

Status. ExecuTorch is primarily an **inference** runtime. On-device training was added as an **experimental** capability starting with the 0.4 release, with the announcement language being "Added experimental capabilities for on-device training, along with an example prototype for LLM fine-tuning." It is not on the stability rails of the inference runtime. The training extension lives under `executorch/extension/training/` and produces a separate `.pte` whose Method graph contains forward + backward.

How it works conceptually. The AOT side uses `torch.export` on an `nn.Module` that has been transformed (via a training-aware wrapper, similar to `functorch/torch.func`) so that the backward pass is materialized as an explicit subgraph. The resulting program has two callable methods: `forward` (returns loss) and a method that returns the gradients (or applies an optimizer step). On-device, the runtime executes both. Optimizer state and the parameters being updated are exposed as mutable buffers and persisted out via the same `register_buffer` mechanism used for KV-caches.

Gradient checkpointing. Standard PyTorch `torch.utils.checkpoint.checkpoint` is recognized at export time and produces a graph where intermediate activations are recomputed during backward rather than stored. This is the single biggest memory-saving lever for on-device LoRA on phones — activations during backward of a 3 B transformer can otherwise easily exceed available RAM. The tradeoff is roughly 33% extra compute for ~3-10× lower activation memory, depending on which transformer blocks are checkpointed.

Layer-by-layer fragmentation. A pattern documented in research papers that build on ExecuTorch (e.g., MobiZO / P-RGE, arXiv 2409.15520) is to fragment the model into per-layer `.pte` programs that are loaded, run forward+backward, parameter-updated, then unloaded sequentially. This bounds peak memory at roughly (one transformer block's weights + activations + LoRA adapter + optimizer state for the LoRA params). For a 1.1 B TinyLlama or 7 B Llama-2 on a Snapdragon-class Android phone with the QNN backend, this is the only practical way to fit fine-tuning into RAM. Note this fragmentation is an *application-level* pattern; ExecuTorch provides the primitives (multiple Methods per Program, `Module` API to share Programs across instances, mutable buffers for state) but does not orchestrate it for you. (arXiv)

LoRA specifics. The LoRA adapter is just two extra Linear layers (down-projection rank `r`, up-projection back to hidden dim) on the side of each attention projection (commonly `q_proj`, `v_proj`, sometimes also `k_proj`, `o_proj`), and the MLP (`gate/up/down`). Only the adapter weights are trainable; the base model weights are frozen (and can stay in int4-quantized form on disk and in HTP memory). After training, adapters can be merged back into the base weights for inference-cost-free deployment, or kept as separate `.pte` segments and swapped at runtime for multi-task scenarios. (TrueFoundry)

Caveat for the audience. As of late 2023 the official runtime docs explicitly said "the ExecuTorch runtime only supports model inference, and does not yet support training" — that language has now been superseded by experimental training support, but Q&A is likely to surface this. The right answer is: training is supported as an experimental extension, with LoRA being the headline use case; production-grade on-device training on Android is still rough at the edges (op coverage for backward kernels, especially under HTP, is the practical bottleneck — most training work today happens on XNNPACK CPU or via zeroth-order/gradient-free methods like MobiZO that avoid needing a backward pass through the inference engine at all). (PyTorch) (arXiv)

6. Backend Delegates: HTP, XNNPACK, Vulkan — when to use which

ExecuTorch ships with 12+ backends. The three relevant to an Android-focused EdgeMind talk:

XNNPACK (CPU). The default and always-available delegate on mobile. Library of highly-optimized neural network operators for ARM/x86/WASM. The `XnnpackPartitioner` walks the graph and pulls out convolutions, linear layers, activations, elementwise ops, and (importantly) **int8 dynamic-activation + int4 group-wise weight** matmuls — these are accelerated via the **KleidiAI** kernels Arm contributed (specifically why Llama 3.2 1B hits 50 tokens/s decode on a OnePlus 12 CPU). XNNPACK requires channels-last memory layout, so the partitioner inserts permute ops at the boundaries; a layout-canonicalization pass minimizes those. Use this when you don't have NPU access, when fallback ops are needed, or when you want a single binary that works across all Android devices.

Vulkan (mobile GPU). Delegate under `backends/vulkan/`. Use this when the device has a strong GPU (Mali, Adreno, Xclipse) but no first-party NPU support, or when you want to free up CPU/NPU for other work. Coverage is narrower than XNNPACK — most CV models partition cleanly, LLMs are a work-in-progress. Build with `-DEXECUTORCH_BUILD_VULKAN=ON`. Today it ships as a separate `.so` and is not in the default Maven AAR; an open issue (#16256, Dec 2025) tracks bundling qnn + vulkan + xnnpack into a single AAR for the Android UX demo. [PyTorch](#) [GitHub](#)

Qualcomm HTP / QNN. The NPU delegate covered above. Use this on Snapdragon devices when you want the best perf-per-watt — HTP is typically 5-10× more energy-efficient than CPU for the same throughput on quantized workloads. Cost: tied to QNN SDK shipped libs (you ship `libQnnHtp*.so`) and Hexagon skel files per Hexagon arch version — v69/v73/v75/v79), `.pte` is SoC-specific (a v75-targeted `.pte` does not run on v69), and op coverage is limited compared to XNNPACK. Current release notes state coverage of **92 out of 116 HTP operators** in the QNN backend — meaning roughly 20% of ops still fall back to CPU, and that boundary crossing is the practical performance limiter for many models.

[GitHub](#)

Fallback behavior. Partitioning is per-subgraph. After the partitioner runs, the graph contains a mix of `executorch_call_delegate` nodes (for delegated subgraphs) and regular `call_function` nodes (for ops that fall back to portable or optimized CPU kernels). At runtime, the executor walks the instruction chain and dispatches each instruction to either a kernel (CPU op) or a backend's `execute()` method. **There is no automatic "if HTP fails, retry on CPU" fallback at runtime** — fallback is decided AOT by the partitioner; runtime delegate execution that fails throws an error. This is a common gotcha: if the QNN context binary errors on a specific device (e.g., a SoC variant the partitioner thought was supported but the on-device QNN library disagrees), you crash, not soft-fall-back. Production code path is: ship two `.pte` files (HTP and XNNPACK), try-load HTP first, fall back to XNNPACK on init failure.

You can also stack partitioners: pass `partitioner=[QnnPartitioner(), XnnpackPartitioner()]` and the QNN partitioner runs first (greedy), then XNNPACK grabs what's left, and anything still unconsumed runs on portable kernels.

7. Phi-3 Mini / Phi-4 Mini on ExecuTorch

Phi-3 Mini. Phi-3 Mini (3.8 B params) was one of the first community-requested models with explicit ExecuTorch support; tracked in issue #3550 and announced as supported via the Hugging Face → ExecuTorch optimum pipeline along with Gemma-2b in the 0.4 release. There is no canonical "official" Phi-3 Mini latency number from the ExecuTorch team because the team's flagship model for benchmarking is Llama 3.2 1B/3B; for Phi-3 Mini int4 you should expect numbers comparable to a similarly-sized model — ONNX Runtime Mobile published Phi-3 Mini RTN-INT4 running "at a moderate speed" on a Samsung Galaxy S21, with two accuracy/perf-tunable RTN levels (`int4_accuracy_level=1` for accuracy, `int4_accuracy_level=4` for performance), and AWQ-INT4 nearly matching FP16 perplexity. ExecuTorch's XNNPACK + KleidiAI path on a modern flagship (8 Gen 3 / SM8650) is in the same ballpark as ORT QNN-EP for similar

quantization, with the LLM benchmarking baseline being Llama 3.2 1B at ~50 tok/s decode / 260 tok/s prefill on OnePlus 12.

Phi-4 Mini (the current flagship Phi model on ExecuTorch). The PyTorch team's official `pytorch/Phi-4-mini-instruct-INT8-INT4` (a `torchao`-quantized variant: 8-bit embeddings, 8-bit dynamic activations, 4-bit weight linears) provides a pre-built `.pte` ready to load. Published numbers:

- **17.3 tokens/sec on iPhone 15 Pro** (decode) `Hugging Face`
- **~3206 MB memory footprint** `Hugging Face`
- Exported with `max_seq_length=1024` by default; re-export to change.

Export flow (XNNPACK):

```
python -m executorch.examples.models.phi_4_mini.convert_weights \
    $(hf download pytorch/Phi-4-mini-instruct-INT8-INT4) \
    pytorch_model_converted.bin
```

A known caveat in the public Hugging Face card: "Our mobile demo apps have regressed support for the Phi-4 tokenizer, so this model will not currently run in our official apps." You can still load the `.pte` from your own runner — a relevant detail for the EdgeMind speaker to highlight given the topic of *custom* orchestration. `Hugging Face`

Phi-4 Mini support is mentioned alongside Llama 3.2/3.1/3, Qwen 3, and LiquidAI LFM2 as the "supported LLMs" set on the main ExecuTorch GitHub README. Note that the edge-LLM leaderboard maintainers rate Phi-4 Mini's ExecuTorch path as "less mature than the Gemma 3 mobile stack" — meaning if you want polished out-of-the-box first-party Android support, Gemma 3 via Google AI Edge is more polished than Phi via ExecuTorch. `GitHub`

`Awesome Agents`

8. ExecuTorch JNI Layer on Android

Packaging. All ExecuTorch Android libraries are shipped as an AAR. As of v1.0+, the canonical way to consume is Maven Central:

```
kotlin

// app/build.gradle.kts
dependencies {
    implementation("org.pytorch:executorch-android:1.0.0")
    // Transitive deps (fbjni, soloader) are pulled via the AAR's pom.
}
```

For direct AAR file usage (snapshot or offline), the AAR does *not* carry pom metadata, so you must declare:

```
kotlin

implementation(files("libs/executorch.aar"))
implementation("com.facebook.soloader:soloader:0.10.5")
implementation("com.facebook.fbjni:fbjni:0.7.0")
```

The AAR contains two ABI variants (`arm64-v8a` and `x86_64`) and bundles: `PyTorch`

- the **Java/Kotlin classes** (`org.pytorch.executorch.Module`, `Tensor`, `EValue`, `NativePeer`, plus `LlamaModule` for LLM use cases);
- `libexecutorch.so`** — the JNI binding (built from `extension/android/`) which wraps `libexecutorch_jni.so`. This contains: core ExecuTorch runtime, XNNPACK backend, Portable kernels, Optimized kernels, Quantized kernels, and LLaMA custom ops.

- For QNN builds, an additional `libqnn_executorch_backend.so` is bundled and you separately ship the QNN SDK shared libraries (`libQnnHtp.so`, `libQnnSystem.so`, `libQnnHtpV75Stub.so`) + the corresponding hexagon skel).

JNI binding stack. ExecuTorch uses `fbjni` (Facebook's lightweight JNI binding helper) rather than hand-rolled JNI. The Java/Kotlin `NativePeer` class holds a `HybridData` (an `fbjni` concept) pointing to the C++ peer object. Lifetimes are managed via `fbjni`'s automatic local/global ref helpers, avoiding the classic JNI ref-leak class of bugs. `SoLoader` (Facebook's library loader, version 0.10.5+) handles the native library load — it's a transitive dep of `fbjni` and is also what triggers the `System.loadLibrary("executorch")` call lazily on first JNI entry.

Typical Kotlin call sequence:

```
kotlin

import org.pytorch.executorch.Module
import org.pytorch.executorch.EValue
import org.pytorch.executorch.Tensor

// Loads libexecutorch.so transitively
val module = Module.load("/data/local/tmp/model.pte")

val input = Tensor.fromBlob(floatArrayOf(...), longArrayOf(1, 3, 224, 224))
val output: Array<EValue> = module.forward(EValue.from(input))
val result: Tensor = output[0].toTensor()
```

`UnsatisfiedLinkError` — the failure modes:

1. **ABI mismatch.** Most common. The app is built `armeabi-v7a`-only or has only `x86_64` and is loaded on an `arm64-v8a` device (or vice versa). Fix: ensure your

`splits.abi` config in `build.gradle` matches the variants in the AAR. ExecuTorch ships only `arm64-v8a` + `x86_64`; armv7 is not supported.

2. **Missing transitive dependencies.** If you direct-load the AAR file (not from Maven), you must explicitly declare `fbjni` and `soloader` dependencies — otherwise `System.loadLibrary("executorch")` will fail because `libfbjni.so` isn't on the linker path.
3. **Missing QNN sidcar libs.** If your `.pte` is QNN-delegated and you forgot to bundle `libQnnHtp.so`, `libQnnHtpV75Skel.so`, etc., into `jniLibs/arm64-v8a/`, you'll get a runtime backend init failure (`[ERROR] [Qnn ExecuTorch]: Cannot Open QNN library libQnnHtp.so`). Note this surfaces during `Module.load()` not `loadLibrary()` — it's a backend init error, not a `UnsatisfiedLinkError`.
4. **Skel/stub version mismatch.** If you push `libQnnHtpV69Skel.so` but your `.pte` was compiled for v75, you'll see "No Snapdragon SOC detected" or a similar backend init error. Each Hexagon arch version (v69 = 8 Gen 1, v73 = 8 Gen 2, v75 = 8 Gen 3, v79 = 8 Gen 4/Elite) needs its own skel/stub pair. [GitHub](#)

Recommended pattern for robust loading:

```
kotlin
```

```

try {
    System.loadLibrary("executorch")
} catch (e: UnsatisfiedLinkError) {
    // Telemetry + fall back to cloud inference
    Log.e("EdgeMind", "Native libs not present for ABI ${Build.SUPPORTED_ABIS[0]}")
}

val module = try {
    Module.load(htePtePath) // Try HTP first
} catch (e: RuntimeException) {
    Module.load(xnnpackPtePath) // Fall back to CPU .pte
}

```

The lower-level JNI binding source lives at `extension/android/jni/jni_layer.cpp` (and `jni_layer_llama.cpp` for the LLM API). Built with CMake driven by `extension/android/CMakeLists.txt`, using NDK r28c at CI (r27b for older releases).

[DeepWiki](#) [PyTorch](#)

9. Model Export Pipeline: `torch.export` → `compile` → `lower` → `.pte`

The full AOT pipeline has three named steps, each with a real Python API surface:

Step 1: `torch.export.export()`. This is PyTorch 2.x's official model-capture API (the successor to TorchScript and `torch.jit.trace`). It runs your `nn.Module.forward()` once with example inputs, captures a single straight-line computation graph in the form of an `ExportedProgram`, and produces ATen IR (a normalized operator set). Critical: models must be in `.eval()` mode (unless training); operations like batch norm / dropout switch behavior. Mutable buffers (KV-caches) must be declared via `register_buffer` and

mutated with in-place ops (e.g., `cache.copy_(new)`), not `cache = new`). Dynamic shapes are expressed via `Dim()` constraints. `PyTorch`

python

```
exported_program = torch.export.export(  
    model, example_inputs,  
    dynamic_shapes={"input": {1: torch.export.Dim("seq", min=1, max=1024)}},  
)
```

Step 2: `to_edge_transform_and_lower()` (the new unified API; older code used `to_edge()` then `to_backend()` separately). This:

1. Lowers ATen IR to **Edge IR** — a smaller, more constrained dialect: dim order is explicit, all ops are core ATen ops (currently ~180 operators in the core set), there are no decomposable composite ops left.
2. Runs any user-specified passes (op replacement, fusion, custom rewriting).
3. Runs the **partitioner(s)** you pass, each of which annotates subgraphs to be delegated to a backend.
4. Calls each backend's `preprocess()` to compile its subgraph into a backend-specific blob (an XNNPACK FlatBuffer, a QNN context binary, a Vulkan shader module set, etc.). These blobs become **delegate segments** in the final `.pte`.
5. Returns an `EdgeProgramManager` (named `program` below).

python

```

from executorch.exir import to_edge_transform_and_lower
from executorch.backends.xnnpack.partition.xnnpack_partitioner import XnnpackPa

program = to_edge_transform_and_lower(
    exported_program,
    partitioner=[XnnpackPartitioner()],
)

```

Step 3: `to_executorch()`. This is where memory planning, instruction emission, and serialization happen:

1. `SpecPropPass` computes a `TensorSpec` (shape, dtype, dim order, lifetime) for every node.
2. `SymShapeEvalPass` resolves any symbolic shape expressions to concrete upper bounds (so static planning can size buffers).
3. **Memory planning algorithm** runs (default: `Greedy`; alternative: `Naive`). Each intermediate tensor gets a fixed offset within a memory arena.
4. **Instruction emission** generates the chain of kernel calls / delegate calls.
5. **FlatBuffer serialization** writes the `Program` + segments.

```

python

executorch_program = program.to_executorch()
with open("model.pte", "wb") as f:
    f.write(executorch_program.buffer)

```

A typical Q&A snag: developers expect a single `.compile()` call. The three-stage split exists because `torch.export` is a generic PyTorch capability that other PT-ecosystem tools (vLLM, AOTInductor) also use — only the latter two steps are ExecuTorch-specific.

10. ExecuTorch Limitations and Known Issues

Operator coverage gaps. The biggest practical limit. The core ATen op set is ~180 ops; ExecuTorch's portable kernels cover them all, but **delegates cover subsets**:

- XNNPACK: very good coverage for CV and quantized transformer blocks, but exotic ops (custom attention, `scatter_*`, some dynamic reductions) fall back to portable CPU.
- QNN/HTP: **92/116 supported HTP operators** as of the most recent release notes — about 20% of ops still drop to CPU. [GitHub](#)
- Vulkan: narrower still; many quantized ops are not yet implemented.
- Ethos-U: int8 fully supported in 1.0, int16 still experimental. [Arm Developer](#)

The practical effect: a model that *exports* fine may still have dozens of `executorch_call_delegate` ↔ CPU boundary crossings, each one demanding a layout transform and a tensor copy. The fix is usually model-side: replace the unsupported op with a delegate-friendly equivalent (e.g., replacing `AdaptiveAvgPool2d` with `torch.mean`) is a documented workaround for a vocal-remover model on XNNPACK). [Arm Developer](#)

Dynamic shapes and data-dependent ops. Ops like `torch.unique_consecutive` or any op with data-dependent output shape (`MaskRCNN`-style NMS) cause real headaches: memory planning can't size their outputs ahead of time. ExecuTorch supports them only via the dynamic-allocation path (the kernel allocates via the `MemoryAllocator` provided at runtime), which means no static memory plan and no AOT optimization for those tensors. Practical guidance: keep data-dependent shapes out of the hot path; do dynamic post-processing in Python/Kotlin code instead.

Mutable buffer fragility. The documented warning: "Export of mutability can be fragile. Consider explicitly calling `detach()` on tensors before assigning to a buffer if you

encounter export-time errors related to gradients. Ensure that any operations done on a mutable buffer are done with in-place operations (typically ending in `()`). KV-cache bugs are the canonical incarnation of this.

No automatic backend fallback at runtime. As discussed above — partitioning is AOT, so a runtime delegate failure crashes rather than gracefully degrading. You ship multiple `.pte`s for resilience.

Tokenizer ergonomics. ExecuTorch's official Phi-4-mini-instruct release card includes the candid warning: "Our mobile demo apps have regressed support for the Phi-4 tokenizer." Tokenizer integration is still a per-model exercise; there's no universal HF-style `AutoTokenizer` on-device. The PyTorch tokenizers project is mentioned in the React Native ExecuTorch roadmap as the direction of travel to reduce bundle size by replacing Hugging Face's heavyweight tokenizers. [Hugging Face](#) [GitHub](#)

Build complexity. Direct quote from Arm's developer blog on porting an ML pipeline to Android: "ExecuTorch is still frequently changing as it is still in beta, this means there are fewer online resources ... less documentation for interfacing with TFLite and ExecuTorch models in C++, making failures harder to diagnose." This was written pre-1.0; the 1.0 GA release (October 2025) explicitly addresses stability/documentation as headline items, but the C++ API surface remains larger than TFLite's.

Debugging tools. ExecuTorch ships:

- **ETDump** — a runtime profiler that records per-op execution times and writes a binary trace; you load it back into a Python analyzer.
- **ETRecord** — an AOT inspection artifact that maps node names in the lowered graph back to source-level Python locations.
- **Bundled programs** — a way to package reference inputs/outputs alongside a `.pte` for on-device correctness validation.
- **executor_runner** — a CLI tool that loads a `.pte` and runs it with logging, very useful for first-line "does my model even load?" diagnosis. Sample log output:

```
I executor_runner.cpp:82] Model file add.pte is loaded.  
I executor_runner.cpp:91] Using method forward  
I executor_runner.cpp:138] Setting up planned buffer 0, size 48.  
I executor_runner.cpp:161] Method loaded.  
I executor_runner.cpp:180] Model executed successfully.
```

Selective build. A useful but unforgiving feature: you can strip unused ops to shrink the binary, but if your model dynamically pulls in an op you didn't declare (rare, but possible with control-flow), you get a runtime error. The selective-build manifest is per-model.

11. Memory Management: Static vs. Dynamic, Arena Allocators

ExecuTorch's memory model is one of its most distinctive features and is the main reason it can target microcontrollers. The runtime requires the **caller** to provide all memory; it never calls `malloc` internally (in heapless mode).

The two allocator categories (from the official C++ runtime tutorial):

1. **Method Allocator** (`MemoryAllocator`). Used at `Method` load time for the runtime's bookkeeping structures: tensor metadata, EValue table, the chain of instructions, delegate handles. This is "init-time" memory. A `MallocMemoryAllocator` variant exists for environments with a heap; for bare-metal you provide a fixed-size byte buffer that the allocator bump-allocates out of.
2. **Planned Memory** (`HierarchicalAllocator`). A collection of one or more memory arenas where **mutable intermediate tensor data** lives. The "hierarchical" part is what lets you put hot, small tensors in SRAM and large, less-frequent ones in DRAM on heterogeneous SoCs (the Hexagon DSP is the classic example: VTCM is fast and small, DDR is large and slow). Each arena is just a `Span<uint8_t>` of memory you own.

The runtime tells you how big to make the arenas:

```
cpp

auto method_meta = program->method_meta("forward");
size_t num = method_meta->num_memory_planned_buffers();
std::vector<std::unique_ptr<uint8_t[]>> planned_buffers;
std::vector<Span<uint8_t>> planned_arenas;
for (size_t id = 0; id < num; ++id) {
    size_t bytes = method_meta->memory_planned_buffer_size(id).get();
    planned_buffers.push_back(std::make_unique<uint8_t[]>(bytes));
    planned_arenas.push_back({planned_buffers.back().get(), bytes});
}
HierarchicalAllocator planned_memory({planned_arenas.data(), planned_arenas.siz
```

In a heapless target, you'd query these sizes once on a desktop, hard-code them into a static byte array, and link the binary — no runtime allocation at all.

Memory planning algorithms (AOT). Three documented passes:

- **SpecPropPass** — annotates each tensor with a **TensorSpec** (shape symbolic expression, dtype, lifetime). (PyTorch)
- **SymShapeEvalPass** — resolves symbolic shapes to upper bounds (required because the planner needs concrete sizes).
- **The planner itself** — two built-in options:
 - **Naive** — concatenates all tensors linearly in memory, no reuse. Serves as an upper bound. Useful as a sanity baseline.
 - **Greedy** (default) — best-fit allocation with lifetime reuse: if a buffer's lifetime doesn't overlap with the new tensor, reuse it; if multiple non-overlapping buffers are candidates, pick the one closest in size to minimize fragmentation. In

practice, Greedy reduces peak memory by 2-5× vs. Naive for typical transformer graphs.

- You can implement your own by subclassing — useful if you have a multi-tier memory hierarchy (e.g., assign attention K/V tensors to SRAM, MLP intermediates to DRAM).

Static vs. dynamic. "Static" memory in ExecuTorch means the planned-memory arenas — fixed sizes computed AOT, no runtime allocation. "Dynamic" applies to ops with data-dependent output shapes, which fall back to using the `MemoryAllocator`'s `allocate()` at runtime. The principle to articulate in Q&A: **dynamic allocation in ExecuTorch is opt-in, op-by-op, and is the exception, not the rule.** This is the central architectural difference from TFLite's interpreter, which routinely calls into a memory pool at runtime.

Constant tensors don't consume planned memory. They point directly into the `mmap()`'d `.pte` file's segment data. So a 3 GB Llama weight blob doesn't sit in your planned arena — it sits in the kernel's page cache and is paged in on demand. This is also why `.pte` segment alignment matters (4096-byte alignment matches the page size for `mmap`).

Delegates can free post-init. "Backend delegates can choose to unload their precompiled data after model initialization, reducing peak memory." Concretely: a QNN delegate parses its serialized context binary in `init()`, hands it to the HTP driver, then is free to drop the on-CPU copy. This is the difference between transient init-time peak memory and steady-state inference memory. `PyTorch`

12. Whisper on ExecuTorch

Status: officially supported, Android demo app exists. Whisper is one of the named supported speech models on the main ExecuTorch README (alongside MobileNetV2, DeepLabV3, YOLO26 for vision). There's an Android Whisper demo at `meta-pytorch/executorch-examples/whisper/android/WhisperApp`. As of the demo's

README, "The ExecuTorch AsrModule API is not yet released. We will give a snapshot AAR soon" — meaning there's an in-progress higher-level Java API for streaming ASR; today you can already build the Whisper (.pte) and run it via the generic Module API.

[GitHub](#)

Export pipeline (Whisper Small). Done via the [optimum-executorch](#) Hugging Face integration:

```
optimum-cli export executorch \  
  --model openai/whisper-small \  
  --task automatic-speech-recognition \  
  --recipe xnnpack \           # use 'cuda' or 'metal' for desktop dev  
  --dtype bfloat16 \  
  --output_dir ./
```

For int4 weight quantization on the encoder (the heavy compute):

```
optimum-cli export executorch \  
  --model openai/whisper-small \  
  --task automatic-speech-recognition \  
  --recipe xnnpack \  
  --dtype bfloat16 \  
  --qlinear 4w \  
  --qlinear_encoder 4w \  
  --qlinear_packing_format tile_packed_to_4d \  
  --qlinear_encoder_packing_format tile_packed_to_4d \  
  --output_dir ./
```

The audio front-end (log-mel spectrogram) is exported as its own separate (.pte):

```
python -m executorch.extension.audio.mel_spectrogram \
    --feature_size 80 \                # 80 for whisper-small/base; 128 for
large-v3
    --stack_output \
    --max_audio_len 300 \              # max 5 minutes
    --output_file whisper_preprocessor.pte
```

You then ship two `.pte`s + the tokenizer (`tokenizer.json`, `tokenizer_config.json`, `special_tokens_map.json`) from the HF repo). ([GitHub](#))

Latency on HTP. No officially published number for Whisper-Small specifically on HTP from the ExecuTorch team that I could verify. Reasonable expectations to set in Q&A based on adjacent data points: Whisper Small is 244 M params, encoder-decoder, and on a Snapdragon 8 Gen 3 / SM8650 HTP with int8 quantization, expected real-time factor (RTF) is well under 0.5× (i.e., 10 seconds of audio transcribed in <5 seconds), bounded mostly by decoder autoregression rather than encoder. On XNNPACK CPU on the same device, RTF is typically 1-2×, varying by language. If a question asks for a specific HTP number, the honest answer is: "Whisper on HTP is supported through QnnPartitioner; ExecuTorch's official latency-published baselines are LLM-focused, so for Whisper you'd want to run the QAI Hub profiling job on your target SoC."

A real-world note from Arm's developer blog: a song-translation pipeline porting effort used **whisper.cpp** (not ExecuTorch Whisper) for the ASR stage because of (a) better C++ runtime maturity at the time and (b) easier JNI integration. EdgeMind's audience may have similar opinions; the current ExecuTorch Whisper story is much improved over what was available in 2024, but whisper.cpp remains a legitimate baseline.

13. ExecuTorch Roadmap and 2025 Updates

The headline 2025 event: ExecuTorch 1.0 GA (October 2025). Announced at PyTorch

Conference 2025 (Oct 22–23, 2025) by Mergen Nachin and Cemal Bilgin (Meta). 1.0 marks the transition from Beta (v0.4, Oct 2024) to General Availability. Concrete things 1.0 brought: `PyTorch`

- **Production stability commitment.** Stable API surface, semver-style guarantees, Maven Central distribution (`org.pytorch:executorch-android:1.0.0`).
- **Broader hardware support.** First-class backends for Apple (Core ML, MPS), Qualcomm (QNN/HTP), Arm (Ethos-U via TOSA, KleidiAI for Cortex-A, CMSIS-NN for Cortex-M), MediaTek (NeuroPilot), Vulkan, XNNPACK. Total of 12+ hardware backends. `GitHub`
- **Arm KleidiAI in XNNPACK by default.** This is what gives Llama 3.2 1B/3B their 2.5×/4.2× speedups on Arm CPUs.
- **TOSA 1.0 alignment** (and TOSA 2025.11.0 by the v1.2 release in late 2025).
- **SME2 support.** On SME2-enabled Arm CPUs (i.e., future flagship 2026 SoCs), Stable Audio Open Small generates 11 seconds of audio in **under 4 seconds**, vs. 7–8 seconds on Neon-only Arm CPUs.
- **100+ models from TorchVision / TorchAudio / HuggingFace running end-to-end on Ethos-U microcontrollers** — including transformer-based architectures like Conformer.
- **Windows support (experimental)** added back in v0.6 and matured into v1.0+.
- **Ready-made Xcode and Android Studio packages** — no source build required for typical use.
- **Native C++ desktop/laptop integration** for productivity-tool style apps using CPU/GPU/NPU on consumer PCs.

v1.2 (most recent stable release as of this writing). Adds:

- TOSA 2025.11.0 update.
- New ops: `fill_.Scalar`, `bitwise_not`, `split_copy`, `remainder`, `copy`, `floor_divide`, `mean`, `sum`, `min/max with unset dim`, `select_scatter`,

`masked_fill_.Scalar`, `clamp.Tensor`, `log1p`, `tan`.

- 6D tensor support and pixel shuffle/unshuffle (super-resolution / image processing models).
- Per-channel rescale support in TOSA. [GitHub](#)
- Coverage of 26 new HF models in OpenVINO backend, including Albert, Bert, DistilBert, Eurobert, Roberta, Gemma3-1B, Llama-3.2-1B/3B-instruct, Qwen2.5-0.5B/1.5B, Qwen3-0.6B/1.7B, Phi-4-mini-instruct, SmolLM2-135M, SmolLM3-3B, T5, ViT variants, EfficientNet, FocalNet, MobileViT/v2, PVT, Swin, Whisper.
- Mixed precision and lookahead decoding for LLMs.
- 92/116 HTP operators supported in the QNN backend. [GitHub](#)
- Automatic installation of model-prep dependencies via the `executorch` pip package.

What's coming (from Arm's published roadmap, since Arm is the most public partner):

- **Dynamic Range Quantization (DRQ)** and **selective quantization** for better accuracy control. [Arm Developer](#)
- **Broader CMSIS-NN support** for Cortex-M.
- **VGF backend** for Arm's upcoming 2026 GPU neural technology.
- **Continued LLM and multimodal LLM coverage** — Voxtral (audio-language), Gemma 3 (vision-language), LFM2 from LiquidAI are all already supported; expect more.

Production credentials cited. Meta's Reality Labs uses ExecuTorch in Meta Ray-Ban Display glasses and Quest 3; Instagram, WhatsApp, Messenger, and Facebook use it for various on-device features (improved video-call quality, music recommendations, creative storytelling). It is referenced as "powering billions of users at Meta with real-time on-device inference" on the main GitHub README. [GitHub](#)

Ecosystem integrations to know about for Q&A:

- **optimum-executorch** — Hugging Face's executorch backend; supports any HF transformer model via `optimum-cli export executorch`.
- **Unsloth** — known LoRA fine-tuning library, now produces ExecuTorch-compatible checkpoints.
- **React Native ExecuTorch** (Software Mansion) — popular React Native binding with its own SDK abstraction; supports Whisper, Qwen 3, Llama 3.2, SmolLM 2, CLIP. They have an active roadmap toward v1.0.0 in 2025.
- **Ultralytics YOLO** — first-party ExecuTorch export support for YOLO11 (and YOLO26 is now in the supported-models list).
- **Cactus** — a third-party hybrid (on-device + cloud fallback) AI engine that competes with raw ExecuTorch for the "I want one API" use case.

Quick Reference: Numbers to Drop in the Q&A

Fact	Value
ExecuTorch core runtime size	< 50 KB
<code>.pte</code> segment alignment	typically 4096 bytes (page-aligned)
Default int4 group size	128
Llama 3.2 1B decode on OnePlus 12	50.2 tok/s
Llama 3.2 1B prefill on OnePlus 12	260 tok/s

Fact	Value
Llama 3.2 1B/3B mobile speedup vs fp16	2.5× decode, 4.2× prefill, 56% size, 41% memory
Phi-4 Mini INT8-INT4 on iPhone 15 Pro	17.3 tok/s, 3206 MB
HTP op coverage in QNN backend	92 / 116
Core ATen ops (approx)	~180
ExecuTorch hardware backends	12+
Maven coordinate	<code>org.pytorch:executorch-android:1.0.0</code>
NDK version (CI)	r28c (latest) / r27b (older)
Required Android transitive deps	<code>com.facebook.soloadersoloaders:0.10.5</code> , <code>com.facebook.fbjni:fbjni:0.7.0</code>
Stable Audio Open Small on SME2	11 s audio in < 4 s
Stable Audio Open Small on regular Arm CPU	11 s audio in 7-8 s
<code>.pte</code> magic bytes	<code>ET</code> + 2 ASCII digits at offset 4; optional <code>eh</code> + 2 digits at offset 8
Supported Hexagon arch versions	v69 (8G1), v73 (8G2), v75 (8G3 / SM8650), v79 (8G4 / Elite)

Fact	Value
Backend partitioner pattern	<code>partitioner=[QnnPartitioner(), XnnpackPartitioner()]</code> for stacked priority
Runtime backend fallback	No automatic runtime fallback — partitioning is AOT

This briefing should be sufficient to handle expert Q&A on every topic the user listed. Three caveats to keep in mind: (a) training support remains experimental, (b) HTP op coverage is incomplete at 92/116, and (c) Phi-3/4-Mini ExecuTorch latency on HTP specifically does not have a first-party published number — Phi-4-Mini's published 17.3 tok/s is on iPhone 15 Pro via the XNNPACK-equivalent path, not Snapdragon HTP.